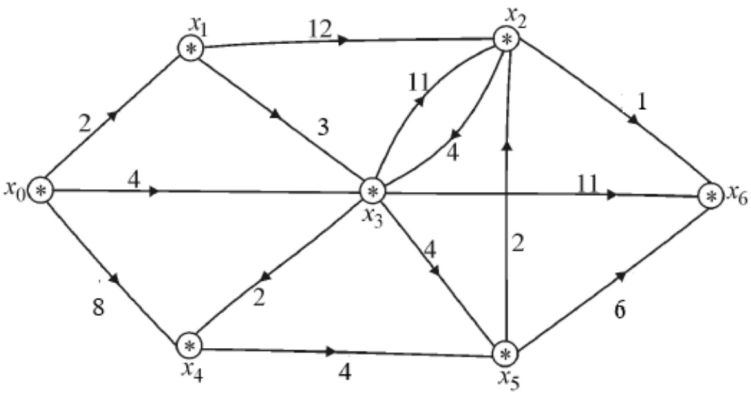
Algoritmica Grafurilor

Tema 3
Iacob Costel, Grupa 106

Exercițiul 1

Problemă

Să se determine drumul de valoare minimă de la x_0 la x_6, în graful din figura următoare, folosind unul din algoritmii de aflare a drumului/drumurilor optim/optime studiați:



Descrierea tuturor nodurilor și a vecinilor + costul din graf:

Nod pornire	Nod Destinație	Cost
x0	x1	2
x0	x3	4
x0	x4	8
x1	x2	12
x1	x3	3
x2	x3	4
x2	x6	1
x3	x2	11
x3	x6	11
x3	x5	4
x3	x4	2
x4	x5	4
x5	x6	6
x5	x2	2

Rezolvarea grafului pentru a găsi cel mai optim drum folosind algoritmul Dijkstra:

Pasul 1: Punct pornire x_0, Cost = 0.
Vecini:
• de la x_0 către x_1 = 2
• de la x_0 către x_3 = 4
• de la x_0 către x_4 = 8

Pasul 2: Aleg x_1 (cel mai mic cost, adică 2), Cost = 2.
Vecini:
• de la x_1 către x_2 = 12 → 2+12 = 14 (E nou, îl notăm)
• de la x_1 către x_3 = 3 → 2+3 = 5 (NU e mai bun decât 4, îl ignorăm)

Pasul 3: Aleg x_3 (următorul cel mai mic cost, adică 4), Cost = 4.
Vecini:
• de la x_3 către x_2 = 11 → 4+11 = 15 (E mai rău decât 14, îl ignorăm)
• de la x_3 către x_6 = 11 → 4+11 = 15 (E nou, îl notăm)
• de la x_3 către x_5 = 4 → 4+4 = 8 (E nou, îl notăm)
• de la x_3 către x_4 = 2 → 4+2 = 6 (E mai bun decât 8, îl actualizăm)

Pasul 4: Aleg x_4 (care acum are costul 6), Cost = 6.
Vecini:
• de la x_4 către x_5 = 4 → 6+4 = 10 (E mai rău decât 8, îl ignorăm)

Pasul 5: Aleg x_5 (care are costul 8), Cost = 8.
Vecini:
• de la x_5 către x_6 = 6 → 8+6 = 14 (E mai bun decât 15, îl actualizăm)
• de la x_5 către x_2 = 2 → 8+2 = 10 (E mai bun decât 14, îl actualizăm)

Pasul 6: Aleg x_2 (care acum are costul 10), Cost = 10.
Vecini:
• de la x_2 către x_3 = 4 → 10+4 = 14 (E mai rău decât 4, îl ignorăm)
• de la x_2 către x_6 = 1 → 10+1 = 11 (E mai bun decât 14, îl actualizăm pentru ultima dată)

Rezultat Final

Drumul reconstruit: x_6 vine din x_2 → x_2 vine din x_5 → x_5 vine din x_3 → x_3 vine din x_0

Drumul minim: $x_0 \to x_3 \to x_5 \to x_2 \to x_6$
Cost total: 4 + 4 + 2 + 1 = 11

Drum optim desenat generat de aplicația Python:

Cod aplicație Python:

```python
import networkx as nx
import matplotlib.pyplot as plt
import heapq

def dijkstra(graph, start, end):
    # Algoritmul Dijkstra pentru calcularea drumului minim
    queue = [(0, start)]
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    shortest_path = {}

    while queue:
        (current_distance, current_node) = heapq.heappop(queue)

        if current_node == end:
            path = []
            while current_node in shortest_path:
                path.append(current_node)
                current_node = shortest_path[current_node]
            path.append(start)
            return path[::-1], distances[end]

        if current_distance > distances[current_node]:
            continue

        for neighbor, weight in graph[current_node].items():
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                shortest_path[neighbor] = current_node
                heapq.heappush(queue, (distance, neighbor))

    return None, float('inf')


def genereaza_graf_complet():
    # 1. Datele grafului
    graph_data = {
        'x0': {'x1': 2, 'x3': 4, 'x4': 8},
        'x1': {'x2': 12, 'x3': 3},
        'x2': {'x3': 4, 'x6': 1},
        'x3': {'x2': 11, 'x6': 11, 'x5': 4, 'x4': 2},
        'x4': {'x5': 4},
        'x5': {'x6': 6, 'x2': 2},
        'x6': {}
    }

    # 2. Calculam drumul optim folosind functia de mai sus
    drum_calculat, cost_total = dijkstra(graph_data, 'x0', 'x6')
    path_edges = list(zip(drum_calculat, drum_calculat[1:]))

    # 3. Pregatim vizualizarea cu NetworkX
    G = nx.DiGraph()
    for node, neighbors in graph_data.items():
        for neighbor, weight in neighbors.items():
            G.add_edge(node, neighbor, weight=weight)

    # Pozitii manuale
```

```python
    pos = {
        'x0': (0, 0),
        'x1': (1, 1),  'x3': (1, 0),  'x4': (1, -1),
        'x2': (2, 0.6), 'x5': (2, -0.6),
        'x6': (3, 0)
    }

    plt.figure(figsize=(13, 8))
    NODE_SIZE = 2200

    # Desenam nodurile
    nx.draw_networkx_nodes(G, pos, node_size=NODE_SIZE,
                           node_color='white', edgecolors='black', linewidths=2)
    nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')

    # 4. Desenam muchiile
    for (u, v) in G.edges():
        is_on_path = (u, v) in path_edges
        color = 'red' if is_on_path else 'black'
        width = 5 if is_on_path else 1.2
        rad = 0.2 if G.has_edge(v, u) else 0.0

        nx.draw_networkx_edges(
            G, pos, edgelist=[(u, v)],
            width=width, edge_color=color,
            arrowstyle='-|>',
            arrowsize=25 if is_on_path else 18,
            connectionstyle=f"arc3,rad={rad}",
            node_size=NODE_SIZE
        )

    # 5. Punem valorile costurilor pe muchii
    edge_labels = nx.get_edge_attributes(G, 'weight')
    for (u, v), w in edge_labels.items():
        x = (pos[u][0] + pos[v][0]) / 2
        y = (pos[u][1] + pos[v][1]) / 2
        if v == 'x2':
            if u == 'x3': y += 0.12
            if u == 'x5': y -= 0.12
        if u == 'x3' and v == 'x6': x += 0.15
        plt.text(x, y, str(w), fontsize=11, fontweight='bold',
                 color='red' if (u, v) in path_edges else 'black',
                 bbox=dict(facecolor='white', edgecolor='none', alpha=0.8),
                 ha='center', va='center')

    plt.title(
        f"Drum optim calculat: {' -> '.join(drum_calculat)} (Cost: {cost_total})",
        fontsize=14
    )
    plt.axis('off')
    plt.show()


if __name__ == "__main__":
    genereaza_graf_complet()
```